



Fine-tune Before Structured Pruning: Towards Compact and Accurate Self-Supervised Models for Speaker Diarization

Jiangyu Han, Federico Landini, Johan Rohdin, Anna Silnova, Mireia Diez, Jan Černocký, Lukáš Burget

Brno University of Technology, Speech@FIT, Czechia

{ihan, landini, rohdin, isilnova, mireia, cernocky, burget}@fit.vut.cz

Abstract

Self-supervised learning (SSL) models like WavLM can be effectively utilized when building speaker diarization systems but are often large and slow, limiting their use in resource-constrained scenarios. Previous studies have explored compression techniques, but usually for the price of degraded performance at high pruning ratios. In this work, we propose to compress SSL models through structured pruning by introducing knowledge distillation. Different from the existing works, we emphasize the importance of fine-tuning SSL models before pruning. Experiments on far-field single-channel AMI, AISHELL-4, and AliMeeting datasets show that our method can remove redundant parameters of WavLM Base+ and WavLM Large by up to 80% without any performance degradation. After pruning, the inference speeds on a single GPU for the Base+ and Large models are 4.0 and 2.6 times faster, respectively. Our source code is publicly available.

Index Terms: speaker diarization, WavLM, fine-tuning, model compression, knowledge distillation, structured pruning

1. Introduction

Recently, several speaker diarization systems [1–3] incorporate WavLM [4], one of the state-of-the-art self-supervised learning (SSL) models, resulting in excellent performance. However, the pre-trained SSL models often have high computational and memory costs at inference time, as well as large storage requirements, making them impractical for real-world deployment.

The challenge posed by large pre-trained models has motivated a growing interest in model compression [5]. One widely used approach is knowledge distillation, where a small student model is trained to mimic the behavior of a large pre-trained teacher model. Prior studies [6–8] have shown promising results with different student architectures. While effective, developing a good student model usually relies on expertise and might lead to suboptimal results. In contrast, structured pruning [9–12] automatically learns a compact architecture by removing the compact sets of redundant parameters, such as attention heads and intermediate dimensions of feed-forward networks, from the large model. Compared to unstructured pruning [13, 14] that removes any individual parameters (i.e. individual weights in a feed-forward layer), structured pruning can lead to actual inference speed-up without support of sparse matrix operations. Besides, independently of the type of pruning, it is known that the performance can be substantially improved by introducing a distillation objective [11, 13, 15].

In the field of speech processing, current efforts mainly focus on compressing pre-trained SSL models for SUPERB [16] evaluation and speech recognition. However, existing methods often lead to significant performance drops at high pruning ra-

tios (e.g., above 70%, i.e. keeping less than 30% of the original parameters) [6, 17, 18], or only report results for low pruning ratios (e.g., under 50%) [19, 20]. Unlike other tasks, speaker diarization systems are usually required to handle long-form conversational recordings, where the real-world acoustic variations, conversational dynamics, and a variable number of speakers complicate the process of model compression. Therefore, efficient compression of pre-trained SSL models for diarization at high pruning ratios without sacrificing performance remains challenging and unsolved, and, to the best of our knowledge, no previous works have focused on this problem.

In this paper, we attempt to compress the SSL models for speaker diarization while keeping the performance unchanged. We choose WavLM [4] and utilize DiariZen [3] to build our diarization pipeline due to its competitive performance. For model compression, structured pruning with knowledge distillation is applied. Given a general SSL model, we first fine-tune the pre-trained model for the diarization task, then remove redundant parameters through structured pruning, and finally, we fine-tune the pruned model again for the diarization task. To verify the effectiveness of our method, we use the far-field single-channel data from AMI [21, 22], AISHELL-4 [23], and AliMeeting [24] for system evaluation. Our experimental results demonstrate that our method can reduce the parameters of both WavLM Base+ and WavLM Large by up to 80% without causing any performance loss. As a result, inference on a single GPU is sped up by 4.0 and 2.6 for the Base+ and Large models, respectively. In addition, our experiments suggest that the Large model always outperforms the Base+ at the same level of sparsity. However, excessive pruning (95% for Large) can severely degrade its performance, allowing the Base+ with an equivalent number of parameters to achieve superior results. Furthermore, we explore the relationship between the similarity of the pruned model to its original teacher and its diarization performance. Along this line, we highlight the importance of fine-tuning SSL models before pruning, which is essential to achieve reasonable results while usually overlooked by previous works [17, 18].

2. Methods

Our method consists of three steps: (1) building a diarization system on top of pre-trained WavLM and fine-tuning it for the diarization task; (2) pruning the fine-tuned model; and (3) further fine-tuning the pruned model using the same diarization pipeline. Figure 1 shows the framework of our method, where the prunable units are individual kernels in the convolutional neural network (CNN) encoder, entire attention heads in the multi-head attention (MHA) blocks, and the rows/columns of weight matrices corresponding to individual intermediate dimensions in the feed-forward network (FFN) blocks.

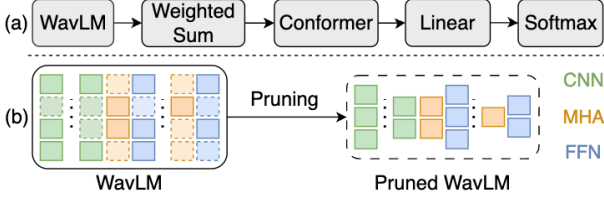


Figure 1: (a) system for WavLM fine-tuning; (b) illustration of structured pruning. The light-colored rectangles will be pruned.

2.1. Speaker diarization pipeline

The speaker diarization pipeline is used for WavLM fine-tuning. We utilized DiariZen [3], a framework with local end-to-end neural diarization (EEND) followed by speaker embedding clustering using pyannote [25], to build our diarization pipeline. The EEND module integrates WavLM [4] with Conformer [26] and is trained using the powerset loss [27]. As suggested in SUPERB [16], the WavLM outputs from each layer are combined using weighted sum to obtain the input sequence for the following Conformer. We follow the original Conformer and use 4 blocks. For each block, the input and hidden dimensions for the feed-forward module are 256 and 1024, respectively; the number of attention heads for the multiheaded self-attention is 4; for the convolution module, the kernel size is set to 31. All dropout rates in the Conformer are set to 0.1. For powerset loss, we assume a maximum of 4 speakers and 2 overlapping speakers. Both WavLM Base+ and WavLM Large are explored, where the number of parameters is 94.4 million and 316.6 million, respectively. For Conformer, the number of parameters is 6.1 million.

2.2. Structured pruning with distillation

As illustrated in Figure 1(b), structured pruning occurs only in WavLM. Following prior studies [9, 11, 18], we consider optimizing the following objective:

$$\mathcal{L}(\theta) + \lambda \|\theta\|_0, \quad (1)$$

where $\mathcal{L}(\theta)$ represents the main training loss, $\|\theta\|_0$ is a regularization term that penalizes the number of non-zero parameters (thus encouraging pruning), and $\lambda > 0$ is a regularization weight controlling the level of pruning. While various choices for the main training loss are possible (see comparisons in the Section 3.3.3), we achieve the best results using a distillation loss inspired by [18]. Specifically, the fine-tuned WavLM is used to initialize both the fixed teacher and the student models, where the student has trainable and prunable parameters θ . Given the t -th frame, i -th Transformer layer outputs of the teacher $\mathbf{h}_t^{(i)}$ and student $\hat{\mathbf{h}}_t^{(i)}$, the distillation loss combines L_1 and cosine distances with equal weights [6, 18]:

$$\mathcal{L}(\theta) = \sum_{i \in S} \sum_{t=1}^T L_1(\mathbf{h}_t^{(i)}, \hat{\mathbf{h}}_t^{(i)} \mathbf{W}_i) - \cos(\mathbf{h}_t^{(i)}, \hat{\mathbf{h}}_t^{(i)} \mathbf{W}_i) \quad (2)$$

where T is the number of frames, and S denotes the set of layers used to match student to teacher via learnable linear transformations $\mathbf{W}_i$. We set $S = \{0, 4, 8, 12\}$ for Base+ models and $S = \{0, 8, 16, 24\}$ for Large models.

Unfortunately, directly optimizing (1) is intractable due to the discrete, non-differentiable nature of the L_0 penalty. Therefore, following [9], we treat θ as a random variable and approximate the original loss (1) by expected value:

$$\min_{\theta, \alpha} \mathbb{E}_{q(\theta|\tilde{\theta}, \alpha)} [\mathcal{L}(\theta) + \lambda \|\theta\|_0] \quad (3)$$

where $q(\theta|\tilde{\theta}, \alpha)$ is a parametric distribution of $\theta = \{\tilde{\theta}_j z_j\}_{j=1}^J$. Here, $\tilde{\theta}_j$ is the j -th group of prunable parameters (e.g., CNN kernels or MHA heads). Each z_j is a random variable following the Hard Concrete distribution $q(z_j|\alpha_j)$ [9], a continuous distribution over interval $[0, 1]$ that assigns non-zero probability mass to $z_j = 0$, which corresponds to pruning (zeroing out) the j -th group of parameters. The parameter $\alpha_j > 0$ controls this probability mass: a smaller α_j increases $q(z_j = 0|\alpha_j)$.

As a result of optimizing (3), we obtain a distribution $q(\theta|\tilde{\theta}, \alpha)$ parametrized by $\tilde{\theta} = \{\tilde{\theta}_j\}_{j=1}^J$ and $\alpha = \{\alpha_j\}_{j=1}^J$. A lower value of α_j corresponds to a higher probability of pruning the j -th parameter group. Based on this distribution, we select a likely value of θ as the parameters for the final pruned model.

Notably, thanks to the Hard Concrete distribution, the term $\mathbb{E}_{q(\theta|\tilde{\theta}, \alpha)} [\|\theta\|_0]$ can be analytically evaluated solely as a function of α , and the gradients of $\mathbb{E}_{q(\theta|\tilde{\theta}, \alpha)} [\mathcal{L}(\theta)]$ can be conveniently approximated using the reparametrization trick. See [9, 11] for further details.

To precisely control sparsity (i.e., the ratio of pruned parameters to the original model size), we adopt an augmented Lagrangian formulation [10] leading to the final loss:

$$\max_{\lambda_1, \lambda_2} \min_{\theta, \alpha} \mathbb{E}_{q(\theta|\tilde{\theta}, \alpha)} [\mathcal{L}(\theta) + \lambda_1 (\|\theta\|_0 - t) + \lambda_2 (\|\theta\|_0 - t)^2]$$

where $\lambda_1, \lambda_2 \in \mathbb{R}$ are learnable Lagrange multipliers, and t is the pre-defined target sparsity.

3. Experiments

3.1. Datasets

We follow the data setups in [3] to use the far-field single-channel data from AMI [21, 22], AISHELL-4 [23], and AI-Meeting [24], for system evaluation. Our model is trained using a combination of the three training sets, with their corresponding development sets combined and used for validation.

3.2. Configurations

For fine-tuning experiments of our method, we utilize the same hyper-parameters as [3] for model training and inference. The maximum number of epochs to further fine-tune the pruned model is 20. The diarization error rate (DER) without collar is used for evaluation, and a macro-averaged DER is reported to represent the overall performance across all datasets.

For pruning and distillation, the model is also trained using the same compound diarization training data. The maximum number of epochs is 30. For the first 5 epochs, target sparsity is linearly increased to the pre-defined values. Afterward, the level of sparsity remains unchanged, and the model parameters continue to update for the rest of the epochs. Following the idea in [18], the pruned student model can be further distilled towards the teacher for up to 20 epochs. Early stopping is applied if the validation loss does not decrease for 5 consecutive epochs. The optimizer is AdamW [28]. We set the learning rate of the main parameters θ to $2e-4$. For parameters of α , λ_1 , and λ_2 , the learning rate is $2e-2$. Our source code is publicly available¹.

3.3. Results and discussion

3.3.1. Overall performance

In Table 1, we show the performance of WavLM models with different levels of sparsity. For reference, we also provide re-

¹<https://github.com/BUTSpeechFIT/DiariZen>

Table 1: *Diarization performance across different far-field datasets. Inference speedups are reported relative to the unpruned model, using a single NVIDIA RTX A5000 GPU with the input batch size optimized for maximum the GPU utilization.*

System	SSL info.			collar=0s			Macro
	Sparsity	#Params.	Speedup	AMI	AISHELL-4	AliMeeting	
Fbank	-	-	-	19.7	12.5	21.0	17.7
WavLM Base+	0%	94.4M	-	15.6	11.8	17.7	15.0
	80%	18.8M	4.0×	15.7	12.1	17.9	15.2
	90%	9.4M	5.7×	17.2	12.1	19.2	16.1
WavLM Large	0%	316.6M	-	14.8	11.3	16.3	14.1
	80%	63.3M	2.6×	15.1	11.3	15.8	14.1
	90%	30.6M	3.5×	15.7	11.2	17.6	14.8

sults using traditional filterbank (Fbank) features as input to the Conformer. As we can see, WavLM-based models attain much better performance than those using Fbank features. For pruning experiments of both WavLM Base+ and WavLM Large, even with 80% of the parameters pruned, our approach delivers performance on par with the unpruned models.

Besides, we also include speedup as an additional metric to highlight the acceleration of inference after pruning. As observed, with 80% sparsity, the inference speed of the Base+ and Large models improves by up to 4.0 and 2.6 times, respectively, and can be further accelerated by removing 90% of the parameters, though at the cost of some loss in performance.

In Figure 2, we compare the absolute inference time for CNN and Transformer layers separately and show them for both unpruned and pruned (80% sparsity) Base+ and Large models. Note that as our aim is to fully utilize GPU, the Base+ model accepts larger batch sizes than the Large model. As we can see, although CNNs contain a small number of parameters (4.2 million, 4.4% for WavLM Base+ and 1.3% for WavLM Large), they are quite resource-intensive and slow during inference, accounting for 35.9% of the total inference time for WavLM Base+ and 22.5% for the WavLM Large. After pruning, it is clear that both models achieve notable and similar acceleration in the Transformer layers. However, the Base+ model eliminates more CNN parameters compared to the Large model, leading to a higher speedup (4.0 vs 2.6) in the end. Since the pruned model is learned in a fully data-driven way, one possible reason for the difference in CNN pruning is that the information encoded in WavLM Large is denser and more informative than in the Base+ model, making it a lower priority for pruning.

3.3.2. Analysis under different sparsities

As suggested in Table 1, leveraging a pre-trained large model consistently proves advantageous. For instance, at 80% sparsity, the pruned WavLM Large contains fewer parameters than the unpruned WavLM Base+ (63.3M vs. 94.4M) while delivering significantly better performance. Note that both WavLM Base+ and WavLM Large are pre-trained using the same corpus, which raises a natural question: if our goal is to obtain a small final model, is it always beneficial to use the largest pre-trained model and then prune it to the desired size?

To answer the above question, as shown in Figure 3, we explore the pruning performance of WavLM Base+ and WavLM Large under different sparsities. As indicated, for the same pruning ratio, the pruned WavLM Large models consistently outperform the pruned WavLM Base+ models. Moreover, both the Large and Base+ models can retain similar performance to the unpruned models even after removing 85% of their parameters. However, the pruned WavLM Large at 94% sparsity, which

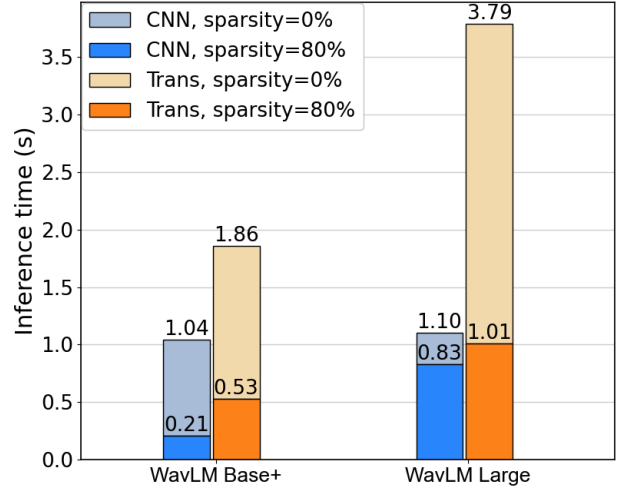


Figure 2: *Inference time comparisons of CNNs and Transformer layers. The averaged values out of 5 runs are reported.*

results in the same number of parameters (18.8M) as the pruned WavLM Base+ at 80% sparsity, performs significantly worse. Our results suggest that overly aggressive pruning can completely undermine a model’s performance. Thus, for scenarios requiring a fast and compact model, pruning from a smaller model may be more reasonable than pruning from a large one.

3.3.3. Ablation analysis

We conduct an ablation analysis of WavLM Base+ in Table 2. The unpruned results are given in the first row as a reference. Rows 2 to 6 compare different pruning strategies, where each pruned model will be further fine-tuned to the diarization task. Specifically, rows 2 and 3 (diar+pruning) show results for models trained using an combination of powerset loss [27] and L_0 norms, following the diarization pipeline in Figure 1(a). Rows 4 and 5 (distill+pruning) show results for pruned WavLM models trained using the distillation loss and pruning loss. The final row reports results after further distillation up to 20 epochs (without pruning) of the pruned model from the original teacher, followed by fine-tuning for diarization. As we can see, directly applying the diarization loss and pruning loss is challenging to achieve satisfactory performance. When applying distillation loss during pruning, it is crucial to initialize both the teacher and student models with WavLM fine-tuned for the diarization task (preFT) to preserve unpruned performance on AMI. However, the performance on AliMeeting remains significantly worse than the unpruned model. This gap can be bridged by further distilling the pruned model to the original teacher.

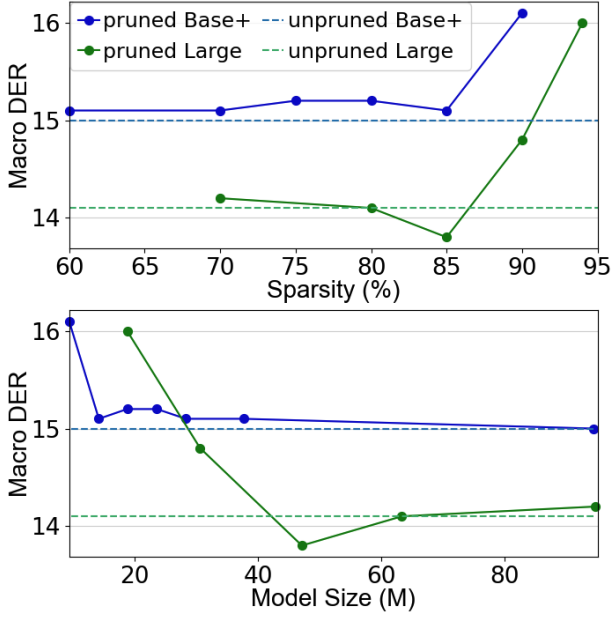


Figure 3: Performance comparisons between WavLM Base+ and WavLM Large under different pruning setups.

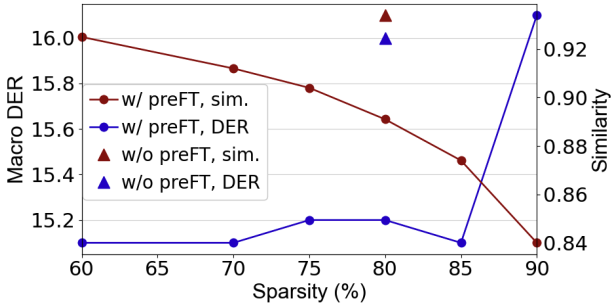


Figure 4: Macro-averaged DER and cosine similarity when using WavLM Base+ under different pruning setups. The triangle points show the performance for 80% sparsity when distilling knowledge from the original (not fine-tuned) WavLM.

3.3.4. Similarity analysis

In Figure 4, we analyze the cosine similarity between the pruned model and its teacher, as well as the DER results across various pruning sparsities. The cosine similarity is averaged over layers of {0, 4, 8, 12} and evaluated on the whole dev data. As observed, when using the fine-tuned WavLM Base+ as a teacher, the cosine similarity decreases as sparsity increases, but it still stays fairly high even for the sparsity of 90%. Furthermore, a lower cosine similarity does not result in worse performance when the sparsity is below 85%. However, when sparsity exceeds 85%, the cosine similarity tends to have a similar pattern to DER performance, as both start to deteriorate significantly.

When using an original WavLM Base+ as a teacher, as indicated by the triangle points, although the pruned model shows high similarity to its teacher, the DER performance is much worse. This result highlights the importance of fine-tuning before pruning. Without this, the pruned WavLM cannot be fine-tuned to achieve good performance on downstream tasks.

3.3.5. Visualization

We visualize how the number of parameters in CNNs, MHAs, and FFNs of the pruned WavLM Base+ are changed in Figure

Table 2: Ablation analysis of pruned WavLM Base+ when sparsity is set to 80%. preFT means fine-tuning the WavLM model to diarization task before pruning. All pruned models will be further fine-tuned to the same diarization task.

System	preFT	collar=0s			Macro
		AMI	AISHELL-4	AliMeeting	
unpruned	-	15.6	11.8	17.7	15.0
diar + pruning	-	16.7	12.0	18.3	15.7
	✓	17.0	12.1	19.7	16.3
distill + pruning	-	16.5	11.9	19.5	16.0
	✓	15.7	12.1	19.1	15.6
+ further distill	✓	15.7	12.1	17.9	15.2

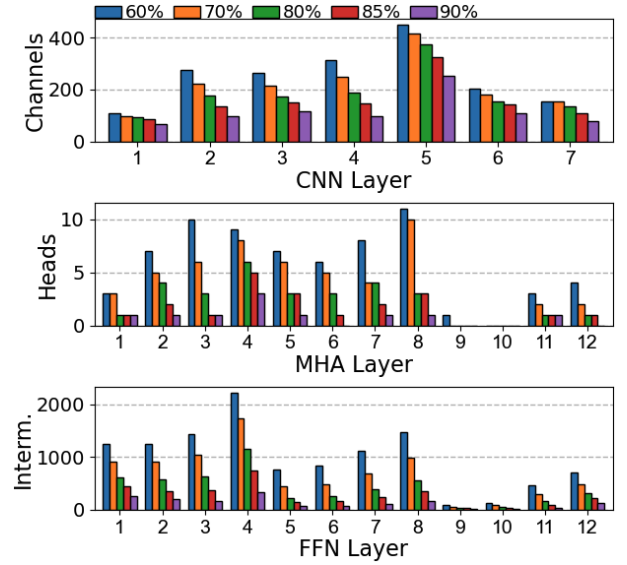


Figure 5: Visualizations of CNN channels, attention heads, and FFN intermediate dimensions from the pruned WavLM Base+. The original sizes are 512, 12, and 3072, respectively. The pruning sparsity is set to {60%, 70%, 80%, 85%, 90%}.

5. As observed, CNN layers in the middle are more relevant. For Transformer layers, the 9th and 10th are almost completely removed after pruning. For most prunable units, higher sparsity generally results in more elements being eliminated proportionally. However, for a few units, such as the 1st and last two layers of MHAs, the structure remains unchanged after a certain level of pruning, suggesting that the current structure is already dense and cannot be further pruned.

4. Conclusion

In this study, we propose compressing SSL models like WavLM through structured pruning, with an application to speaker diarization. The experiments on multiple real far-field datasets prove that our method achieves strong performance. After pruning, our method can keep only 20% of the parameters while maintaining the same diarization performance as with the original model. By doing so, an actual inference acceleration is also achieved without the need for specialized sparse matrix operations. We performed a comprehensive analysis and emphasized the importance of fine-tuning pre-trained SSL models before structured pruning. Additionally, we found that pruning the largest model is not always optimal for some scenarios. We released our code for reproducibility.

5. Acknowledgements

The work was supported by Ministry of Education, Youth and Sports of the Czech Republic (MoE) through the OP JAK project "Linguistics, Artificial Intelligence and Language and Speech Technologies: from Research to Applications" (ID:CZ.02.01.01/00/23_020/0008518), by European Defence Fund project ARCHER, and Horizon 2020 Marie Skłodowska-Curie grant ESPERANTO, No.101007666. Computing on IT4I supercomputer was supported by MoE through the e-INFRA CZ (ID:90254).

6. References

- [1] N. Tawara, M. Delcroix, A. Ando, and A. Ogawa, "NTT speaker diarization system for CHiME-7: multi-domain, multi-microphone end-to-end and vector clustering diarization," in *Proc. ICASSP*. IEEE, 2024, pp. 11 281–11 285.
- [2] A. Plaquet, N. Tawara, M. Delcroix, S. Horiguchi, A. Ando, and S. Araki, "Mamba-based segmentation model for speaker diarization," *arXiv preprint arXiv:2410.06459*, 2024.
- [3] J. Han, F. Landini, J. Rohdin, A. Silnova, M. Diez, and L. Burget, "Leveraging self-supervised learning for speaker diarization," in *Proc. ICASSP*. IEEE, 2025, pp. 1–5.
- [4] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [5] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [6] H.-J. Chang, S.-w. Yang, and H.-y. Lee, "DistilHuBERT: Speech representation learning by layer-wise distillation of hidden-unit bert," in *Proc. ICASSP*. IEEE, 2022, pp. 7087–7091.
- [7] T. Ashihara, T. Moriya, K. Matsuura, and T. Tanaka, "Deep versus wide: An analysis of student architectures for task-agnostic knowledge distillation of self-supervised speech models," in *Proc. Interspeech*, 2022, pp. 411–415.
- [8] S. Gandhi, P. von Platen, and A. M. Rush, "Distil-Whisper: Robust knowledge distillation via large-scale pseudo labelling," *arXiv preprint arXiv:2311.00430*, 2023.
- [9] C. Louizos, M. Welling, and D. P. Kingma, "Learning sparse neural networks through L_0 regularization," in *Proc. ICLR*, 2018.
- [10] Z. Wang, J. Wohlwend, and T. Lei, "Structured pruning of large language models," in *Proc. EMNLP*, 2020, pp. 6151–6162.
- [11] M. Xia, Z. Zhong, and D. Chen, "Structured pruning learns compact and accurate models," in *Proc. ACL*, 2022, pp. 1513–1528.
- [12] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared LLaMA: Accelerating language model pre-training via structured pruning," in *Proc. ICLR*, 2024.
- [13] V. Sanh, T. Wolf, and A. Rush, "Movement pruning: Adaptive sparsity by fine-tuning," *Advances in neural information processing systems*, vol. 33, pp. 20 378–20 389, 2020.
- [14] S. Huang, D. Xu, I. E. Yen, Y. Wang, S.-E. Chang, B. Li, S. Chen, M. Xie, S. Rajasekaran, H. Liu *et al.*, "Sparse progressive distillation: Resolving overfitting under pretrain-and-finetune paradigm," in *Proc. ACL*, 2022, pp. 190–200.
- [15] F. Lagunas, E. Charlaix, V. Sanh, and A. M. Rush, "Block pruning for faster transformers," in *Proc. EMNLP*, 2021, pp. 10 619–10 629.
- [16] S.-w. Yang, P.-H. Chi, Y.-S. Chuang, C.-I. J. Lai, K. Lakhotia, Y. Y. Lin, A. T. Liu, J. Shi, X. Chang, G.-T. Lin *et al.*, "SUPERB: Speech processing universal performance benchmark," in *Proc. Interspeech*, 2021, pp. 1194–1198.
- [17] H. Wang, S. Wang, W.-Q. Zhang, H. Suo, and Y. Wan, "Task-agnostic structured pruning of speech representation models," in *Proc. Interspeech*, 2023, pp. 231–235.
- [18] Y. Peng, Y. Sudo, S. Muhammad, and S. Watanabe, "DPHuBERT: Joint distillation and pruning of self-supervised speech models," in *Proc. Interspeech*, 2023, pp. 62–66.
- [19] Y. Peng, K. Kim, F. Wu, P. Sridhar, and S. Watanabe, "Structured pruning of self-supervised pre-trained models for speech recognition and understanding," in *Proc. ICASSP*. IEEE, 2023, pp. 1–5.
- [20] H. Jiang, L. L. Zhang, Y. Li, Y. Wu, S. Cao, T. Cao, Y. Yang, J. Li, M. Yang, and L. Qiu, "Accurate and structured pruning for efficient automatic speech recognition," in *Proc. Interspeech*, 2023, pp. 4104–4108.
- [21] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaikos, W. Kraaij, M. Kronenthal *et al.*, "The AMI meeting corpus: A pre-announcement," in *International workshop on machine learning for multimodal interaction*. Springer, 2005, pp. 28–39.
- [22] W. Kraaij, T. Hain, M. Lincoln, and W. Post, "The AMI meeting corpus," in *Proc. International Conference on Methods and Techniques in Behavioral Research*, 2005, pp. 1–4.
- [23] Y. Fu, L. Cheng, S. Lv, Y. Jv, Y. Kong, Z. Chen, Y. Hu, L. Xie, J. Wu, H. Bu *et al.*, "AISHELL-4: An open source dataset for speech enhancement, separation, recognition and speaker diarization in conference scenario," in *Proc. Interspeech*, 2021, pp. 3665–3669.
- [24] F. Yu, S. Zhang, Y. Fu, L. Xie, S. Zheng, Z. Du, W. Huang, P. Guo, Z. Yan, B. Ma *et al.*, "M2MeT: The ICASSP 2022 multi-channel multi-party meeting transcription challenge," in *Proc. ICASSP*. IEEE, 2022, pp. 6167–6171.
- [25] H. Bredin, "pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe," in *Proc. Interspeech*, 2023, pp. 1983–1987.
- [26] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech*, 2020, pp. 5036–5040.
- [27] A. Plaquet and H. Bredin, "Powerset multi-class cross entropy loss for neural speaker diarization," in *Proc. Interspeech*, 2023, pp. 3222–3226.
- [28] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.